

PLANO DE VERIFICAÇÃO (VP)

Battery Management System (BMS) Digital

CI Digital – CEPEDI –Trabalho Orientado

Sara Ferreira e Samira Freitas

Versão	Data	Descrição
1.0	18/09/2026	Versão inicial do plano de verificação (simplificada)

Siglas

Sigla	Significado
BMS	Battery Management System (sistema de gerenciamento de bateria)
DUT	Design Under Test (projeto sob teste)
VP	Verification Plan (plano de verificação)
FSM	Finite State Machine (máquina de estados)
ULA	Unidade Lógica e Aritmética
SOC	State of Charge (estado de carga)
OV / UV / OT / LK	Sobretensão / Subtensão / Sobretemperatura / Sobrecorrente
BFM	Bus Functional Model (modelo funcional de barramento, ex.: mestre I2C)

1. Introdução

1.1 Visão geral do projeto

O projeto consiste em um BMS (sistema de gerenciamento de bateria) digital descrito em Verilog, voltado a um pack de 4 células de lítio ligadas em série. O sistema recebe as leituras já digitalizadas (10 bits) de tensão de cada célula, corrente e temperatura, compara esses valores com limites de operação configuráveis e, a partir disso, detecta falhas, habilita ou bloqueia a carga e a descarga, comanda o balanceamento das células e estima o estado de carga (SOC) por contagem de Coulomb.

1.2 O que é um plano de verificação

De acordo com o material “Introdução à Verificação Funcional”, o plano de verificação define o que precisa ser verificado e como isso será verificado; ele funciona como a especificação do próprio ambiente de verificação. O artigo “An Overview of Static and Simulation-Based Techniques for SoC Verification” reforça que o VP é o ponto de partida da verificação funcional e o documento mais importante para a equipe, pois define as features a testar, as ferramentas e estratégias, e as métricas de progresso e de conclusão.

Passos para elaboração:

O plano foi montado seguindo as quatro fases apresentadas no material:

- **1. Definir as metas:** foram escritos os objetivos (o que precisa funcionar) e os critérios de conclusão (quando parar), como 100% das features testadas e 90% de cobertura de código.
- **2. Levantar a lista de features:** o código Verilog de cada módulo do projeto foi lido (FSM, ULA, ROM, I2C, balanceamento, SOC etc.) e cada comportamento observável virou uma linha da tabela 2.1.
- **3. Planejar o ambiente:** foram aproveitados os três testbenches já existentes (geral, dataset e reativo) e indicado o que falta (aleatório-restrito e assertions).
- **4. Listar os testes:** para cada feature foi dito como gerar o estímulo e qual o critério de cobertura, no mesmo formato da tabela de exemplo da UART do material.

1.3 Descrição do DUT

O DUT é o módulo de topo bms_top, que integra os seguintes blocos:

Bloco (arquivo)	Função
bms_regs_entrada	Registradores que capturam V1–V4, I e T quando a FSM ativa load_V/I/T.
bms_rom	Guarda os limites (OV=420, UV=300, I _{max} =80, T _{max} =60, capacidade=500 após reset); pode ser reescrita via I2C.
bms_i2c_slave_config	Escravo I2C (endereço 0x42) que recebe registrador + dado de 16 bits e grava na ROM.
bms_mux_A / bms_mux_B	Selecionam o dado medido (A) e o limite (B) que vão para a ULA.
bms_ula	Executa ADD, SUB, CMP (A>B), PAS, CLT (A<B) e BAL (sinaliza hora do balanceamento).
bms_reg_status	Guarda as flags de falha OV, UV, OT e LK (ficam presas em 1 até o reset).
bms_fsm	Máquina de 8 estados que sequencia leitura, checagens, balanceamento/SOC e falha.
bms_control_potencia	Gera CHG_EN e DSCHG_EN a partir do estado, das flags e de CHG_PWR_GD.
bms_control_balanceamento	Encontra v _{min} e liga BAL_EN_x das células acima de v _{min} + 10.
bms_soc_coulomb	Calcula o SOC (0 a 1000) por contagem de Coulomb a cada varredura.

Interfaces externas do DUT:

- **Entradas:** sys_clk, sys_rst (assíncrono), CHG_PWR_GD, I_DIR, V1_dig a V4_dig, I_dig, T_dig (10 bits cada), I2C_SCL e I2C_SDA (bidirecional).
- **Saídas:** CHG_EN, DSCHG_EN, OV_FLG, UV_FLG, OT_FLG, LK_FLG, bal_cmd_out[3:0], BAL_FLG, SOC_DATA_OUT[9:0] e Estado_atual[2:0].

Fluxo da FSM (um estado por ciclo de clock): ST_INIT → ST_READ_SENSORS → ST_CHECK_OV (4 células) → ST_CHECK_UV (4 células) → ST_CHECK_OT →

ST_CHECK_LK → ST_CALC_BAL_SOC → volta para ST_READ_SENSORS. Uma varredura completa leva cerca de 13 ciclos. Se qualquer flag de falha for ativada, a FSM vai para ST_FAULT e só sai com reset.

1.4 Objetivos e metas da verificação

Objetivo geral: demonstrar, por simulação, que o bms_top se comporta conforme a especificação em condições normais, nos limites de operação e em situações de falha, antes de seguir para síntese/implementação.

Metas específicas:

- Verificar que cada proteção (OV, UV, OT, LK) é ativada quando o valor ultrapassa o limite e não é ativada quando está dentro do limite, incluindo os valores exatamente no limite (casos de borda).
- Verificar que a FSM percorre a sequência correta de estados, entra em ST_FAULT diante de qualquer falha e permanece nele até o reset.
- Verificar que CHG_EN e DSCHG_EN são coerentes com o estado da FSM, as flags e CHG_PWR_GD.
- Verificar o balanceamento: células acima de $v_{min} + BAL_DELTA$ devem ter $BAL_EN = 1$ e BAL_FLG deve sinalizar isso.
- Verificar o cálculo do SOC na carga e na descarga, incluindo saturação em 0 e em 1000, comparando com um modelo de referência.
- Verificar a escrita dos limites via I2C e que os novos limites passam a valer nas comparações.
- Garantir que o reset leva todos os registradores ao valor inicial esperado.

Critérios de conclusão (quando a verificação é considerada terminada):

- 100% das features da tabela da Seção 2.1 com pelo menos um teste passando.
- Todos os testes da regressão executando sem nenhuma mensagem [FALHA].
- Cobertura funcional: todos os estados e transições válidas da FSM visitados; cada flag ativada e não ativada; cada célula testada como a maior e a menor tensão.
- Cobertura de código (linha/ramo) de pelo menos 90%, com as exceções justificadas.
- Todos os bugs encontrados registrados e fechados (ou justificados).

Escopo e limitações: a verificação é funcional, em nível de RTL, por simulação (Icarus Verilog + GTKWave/Python para visualização). Não fazem parte deste plano: temporização pós-síntese, consumo, o front-end analógico (conversores A/D) e a precisão elétrica do modelo de bateria.

2. Plano de teste

2.1 Features a serem verificadas

Cada linha abaixo segue o formato de item de plano visto no material: feature, o que verificar, como gerar o estímulo e o critério de cobertura.

ID	Feature	O que verificar	Como (estímulo)	Critério de cobertura
----	---------	-----------------	-----------------	-----------------------

F1	Reset	Após sys_rst: flags = 0, SOC = 1000, FSM em ST_INIT, CHG_EN = DSCHG_EN = 0, ROM com valores padrão.	Dirigido: pulso de reset no início e no meio da operação (inclusive em ST_FAULT).	Reset aplicado em pelo menos 3 estados diferentes da FSM.
F2	Sequência da FSM	Ordem dos estados, contagem das 4 células em CHECK_OV/UV e retorno a READ_SENSORS.	Dirigido: operação normal sem falhas por várias varreduras.	Todos os estados e transições válidas visitados.
F3	Sobretensão (OV)	OV_FLG = 1 se $V_x > \text{lim_sobrecarga}$; 0 se $V_x \leq \text{limite}$.	Dirigido: cada célula em 430, 421, 420 e 419. Aleatório-restrito: V em torno do limite.	Cada célula acima, no e abaixo do limite.
F4	Subtensão (UV)	UV_FLG = 1 se $V_x < \text{lim_sobredescarga}$.	Dirigido: cada célula em 250, 299, 300 e 301. Dataset NASA (descarga real).	Cada célula acima, no e abaixo do limite.
F5	Sobret temperatura (OT)	OT_FLG = 1 se $T > \text{lim_temp}$.	Dirigido: T = 75, 61, 60 e 59.	Acima, no e abaixo do limite.
F6	Sobrecorrente (LK)	LK_FLG = 1 se $I > \text{lim_corrente_fuga}$.	Dirigido: I = 100, 81, 80 e 79.	Acima, no e abaixo do limite.
F7	Estado de falha	Qualquer flag leva a ST_FAULT; flags ficam presas; só o reset limpa.	Dirigido: provocar cada falha isoladamente e duas ao mesmo tempo.	Entrada em ST_FAULT por cada uma das 4 flags.
F8	Controle de potência	CHG_EN = CHG_PWR_GD e não OV; DSCHG_EN = não UV; ambos 0 em INIT/FAULT, OT ou LK.	Dirigido: combinar CHG_PWR_GD = 0/1 com cada estado/flag.	Tabela-verdade de CHG_EN e DSCHG_EN completa.
F9	Balanceamento	BAL_EN_x = 1 para células com $V > v_{\text{min}} + 10$; BAL_FLG = OU dos bits; atualiza só em ST_CALC_BAL_SOC.	Dirigido: V = 380/395/385/400. Aleatório-restrito: 4 tensões entre 300 e 420.	Cada célula como v_{min} ; diferença de 9, 10 e 11 para v_{min} ; nenhuma e todas balanceando.
F10	SOC (Coulomb)	$\Delta = I \cdot 1000 / \text{cap}$; soma na carga, subtrai na descarga, satura em 0 e 1000; cap = 0 não altera o SOC.	Dirigido: I = 40 carga/descarga. Modelo de referência no testbench. Dataset NASA.	Carga, descarga, saturação em 0, saturação em 1000 e cap = 0.
F11	Configuração via I2C	Escrita nos endereços 0x00 a 0x05 atualiza o limite certo; endereço I2C errado é ignorado.	BFM mestre I2C (tasks i2c_start/stop/write_byte) com escritas dirigidas e aleatórias.	Todos os 6 registradores escritos; um endereço inválido.
F12	Operação em malha fechada	O BMS reage sozinho à bateria: corta carga/descarga ao violar limites e balanceia.	Modelo reativo bms_battery_model (carregador e carga plugados).	Ciclo completo de carga e descarga até a proteção atuar.

Pontos de atenção levantados na análise do RTL (devem virar testes específicos):

- Em `bms_control_balanceamento`, `bal_cmd_calc` não recebe valor quando `v_min = 0`, o que gera latch; testar o caso com uma célula em 0.
- Há divergência nos comentários sobre `I_DIR` (no topo: 1 = diminui o SOC; em `bms_soc_coulomb`: 1 = carregando). O teste F10 deve confirmar qual é o comportamento correto pela especificação.
- O limite `lim_corrente_min` (0x03) é gravado, mas não é usado em nenhuma comparação; confirmar se isso é intencional.
- A falha é detectada no ciclo seguinte à comparação; os testes devem esperar uma varredura completa antes de checar a flag.

2.2 Estratégias de inserção de estímulos

Seguindo o material de estudo, o estímulo pode ser dirigido, aleatório ou aleatório-restrito. Para este projeto serão usadas quatro formas complementares:

- **Testes dirigidos (directed tests):** valores escolhidos à mão para exercitar uma feature por vez, principalmente os casos de borda dos limites (ex.: 419, 420, 421). Já implementados em `bms_testbench_geral.v` (Testes 1 a 6).
- **Dados reais (dataset NASA):** as amostras reais de uma bateria de lítio são convertidas por `gerarDados.py` em `battery_data.mem` e aplicadas ao DUT em `bms_testbench_dataset.v`. Isso gera cenários realistas, como a descarga que termina acionando a proteção UV.
- **Simulação reativa (modelo de bateria):** em `tb_bms_reativo.v`, o modelo `bms_battery_model` recebe `CHG_EN`, `DSCHG_EN` e `bal_cmd_out` do BMS e devolve novas tensões, corrente e temperatura, fechando o laço. Um pulso `preset_en` permite impor casos específicos.
- **Aleatório-restrito (a implementar):** geração com `$urandom_range` (ou classes com constraints em SystemVerilog) de tensões, correntes e temperaturas concentradas perto dos limites, com várias sementes, para achar casos que os testes dirigidos não previram.

Para a interface I2C é usado um BFM simples (tasks do testbench que geram START, bytes, ACK e STOP), reaproveitável em todos os testbenches.

2.3 Checagem das respostas

- Autoverificação: a task `check()` compara cada saída com o valor esperado e imprime [OK] ou [FALHA], sem precisar olhar a forma de onda.
- Modelo de referência: um modelo simples do SOC e do balanceamento dentro do testbench prevê o resultado esperado para comparar com o DUT.
- Assertions (a implementar): por exemplo, “se `OV_FLG = 1` então `CHG_EN = 0`” e “em `ST_FAULT`, `CHG_EN = DSCHG_EN = 0`”.
- Inspeção visual: arquivos VCD (GTKWave) e gráficos gerados por `plot_bms.py`, usados para depuração.

3. Cronograma preliminar

Cronograma estimado em semanas a partir da aprovação deste plano. As revisões seguem o ciclo de três passos (inicial, intermediária e final) visto no material.

Semana	Atividade	Entregável / marco
1	Revisão da especificação, levantamento das features e escrita do VP	VP v1.0 – revisão inicial
2	Ajuste do ambiente: tasks comuns (check, BFM I2C, reset), script de compilação e regressão	Ambiente base funcionando (bring-up: reset + 1 varredura)
3	Testes dirigidos F1 a F8 (reset, FSM, falhas e potência), incluindo casos de borda	Testes dirigidos passando
4	Testes F9 a F11 (balanceamento, SOC com modelo de referência, I2C)	Revisão intermediária do VP
5	Estímulo aleatório-restrito, assertions e simulações com dataset NASA e modelo reativo (F12)	Regressão completa
6	Coleta de cobertura, correção de bugs, fechamento e relatório final	Relatório de verificação – revisão final